UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2023 Algoritmos y Estructuras de Datos
Implementación de diccionarios
Semana 5

Wilder Nina Choquehuayta

wnina@utec.edu.pe

8 de septiembre de 2026

¿Qué aprenderemos hoy?

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura: arreglo de listas enlazadas

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura: arreglo de listas enlazadas
- Implementando `insertar`, `buscar` y `eliminar`

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura: arreglo de listas enlazadas
- Implementando `insertar`, `buscar` y `eliminar`
- Creciendo la tabla: *rehash*

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura: arreglo de listas enlazadas
- Implementando `insertar`, `buscar` y `eliminar`
- Creciendo la tabla: *rehash*
- `std::unordered_map` en la práctica

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La estructura: arreglo de listas enlazadas
- Implementando `insertar`, `buscar` y `eliminar`
- Creciendo la tabla: *rehash*
- `std::unordered_map` en la práctica
- Aplicación: conteo de frecuencias

La estructura

Con qué piezas vamos a construir hoy

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Ya tenemos todo lo necesario

Un `vector` (semana 2) de punteros a nodos de una **lista enlazada** (semana pasada). Hoy los combinamos para construir una tabla hash con encadenamiento.

Con qué piezas vamos a construir hoy

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Ya tenemos todo lo necesario

Un `vector` (semana 2) de punteros a nodos de una **lista enlazada** (semana pasada). Hoy los combinamos para construir una tabla hash con encadenamiento.

La estructura

Un arreglo `buckets` de tamaño m: `buckets[i]` es la **cabeza** de la lista enlazada de ese bucket (o `nullptr` si está vacío).

El nodo: clave, valor y siguiente

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Igual que en una lista simple, con un campo más

Cada nodo guarda una **clave**, un **valor**, y un puntero **siguiente** al próximo nodo del mismo *bucket* (o nulo si es el último).

El nodo: clave, valor y siguiente

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Igual que en una lista simple, con un campo más

Cada nodo guarda una **clave**, un **valor**, y un puntero **siguiente** al próximo nodo del mismo *bucket* (o nulo si es el último).

Exactamente el nodo de una lista simple — solo que ahora guarda un **par** (clave, valor) en vez de un solo dato.

La tabla

Los campos de H

$H.buckets$: un arreglo de m punteros a nodo (uno por *bucket*, inicialmente todos nulos). $H.n$: cuántas claves hay guardadas en total.

La tabla

Los campos de H

$H.buckets$: un arreglo de m punteros a nodo (uno por *bucket*, inicialmente todos nulos). $H.n$: cuántas claves hay guardadas en total.

$H.hash(clave)$

clave mód $|H.buckets|$ — el método de la división de la clase pasada. Para claves *string*, primero se convierten a un entero con hashing polinomial, y luego se aplica el mismo mód.

Insertar, buscar, eliminar

Insertar: pseudocódigo

Algoritmo 1: INSERTAR(H, *clave*, *valor*)

```
idx ← H.hash(clave)
p ← H.buckets[idx]
mientras p ≠ nulo hacer
    si p.clave = clave entonces
        p.valor ← valor; devolver          // ya existía: actualizar
    p ← p.siguiente
nuevo ← nodo con (clave, valor), siguiente ← H.buckets[idx]
H.buckets[idx] ← nuevo
H.n ← H.n + 1
```

Buscar: pseudocódigo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 2: BUSCAR(H, *clave*)

```
idx ← H.hash(clave)
p ← H.buckets[idx]
mientras p ≠ nulo hacer
    si p.clave = clave entonces
        devolver p.valor
    p ← p.siguiente
devolver NO-ENCONTRADO
```

Eliminar: pseudocódigo

Algoritmo 3: ELIMINAR(H, *clave*)

```
idx ← H.hash(clave)
p ← H.buckets[idx];  anterior ← nulo
mientras p ≠ nulo y p.clave ≠ clave hacer
    anterior ← p;  p ← p.siguiente
si p = nulo entonces
    devolver                                  // la clave no existía
si anterior = nulo entonces
    H.buckets[idx] ← p.siguiente
en otro caso
    anterior.siguiente ← p.siguiente
liberar p;  H.n ← H.n − 1
```

Ejemplo trabajado: insertando paso a paso

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$m = 7$, insertamos 23, 45, 12, 37 en ese orden

$h(23)=2$: bucket 2 vacío $\rightarrow$ [23].
$h(45)=3$: bucket 3 vacío $\rightarrow$ [45].
$h(12)=5$: bucket 5 vacío $\rightarrow$ [12].
$h(37)=2$: bucket 2 **ya tiene** a 23 $\rightarrow$ [37 $\rightarrow$ 23] (37 se inserta al inicio).

Ejemplo trabajado: insertando paso a paso

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$m = 7$, insertamos 23, 45, 12, 37 en ese orden

$h(23)=2$: bucket 2 vacío $\rightarrow$ [23].
$h(45)=3$: bucket 3 vacío $\rightarrow$ [45].
$h(12)=5$: bucket 5 vacío $\rightarrow$ [12].
$h(37)=2$: bucket 2 **ya tiene** a 23 $\rightarrow$ [37 $\rightarrow$ 23] (37 se inserta al inicio).

Cuatro inserciones, cada una $O(1)$ (más el recorrido del bucket, vacío o de tamaño 1 en este ejemplo) — la misma tabla final que armamos "a mano" la clase pasada.

Creciendo la tabla

¿Cuándo crecer?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Si nunca aumentamos m, y seguimos insertando claves, ¿qué le pasa al factor de carga $\alpha = n/m$ con el tiempo?

¿Cuándo crecer?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Si nunca aumentamos m, y seguimos insertando claves, ¿qué le pasa al factor de carga $\alpha = n/m$ con el tiempo?

Respuesta

Crece sin límite — y con él, el tiempo esperado $\Theta(1 + \alpha)$ deja de ser $O(1)$. Igual que `vector` y la cola circular: hay que **crecer** la tabla cuando se llena demasiado.

¿Cuándo crecer?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Si nunca aumentamos m, y seguimos insertando claves, ¿qué le pasa al factor de carga $\alpha = n/m$ con el tiempo?

Respuesta

Crece sin límite — y con él, el tiempo esperado $\Theta(1 + \alpha)$ deja de ser $O(1)$. Igual que `vector` y la cola circular: hay que **crecer** la tabla cuando se llena demasiado.

Regla usual

Duplicar m cuando α supera un umbral fijo (por ejemplo, $\alpha \geq 0{,}75$).

Crecer: pseudocódigo

Algoritmo 4: CRECER(H)

viejo $\leftarrow$ $H.buckets$
$H.buckets \leftarrow$ arreglo nuevo de tamaño $2 \cdot |viejo|$, todo nulo
para cada *bucket b de viejo* **hacer**
    $p \leftarrow b$
    mientras $p \neq nulo$ **hacer**
        $siguiente \leftarrow p$.siguiente
        $idx \leftarrow H.hash(p.clave)$ `// con el nuevo tamaño`
        p.siguiente $\leftarrow H.buckets[idx]$
        $H.buckets[idx] \leftarrow p$ `// reutilizamos el nodo, sin new/delete`
        $p \leftarrow siguiente$

¿Cuándo se llama Crecer?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dentro de INSERTAR, justo antes de agregar el nodo nuevo

si $\dfrac{H.n+1}{|H.buckets|} \geq 0{,}75$, primero CRECER($H$).

¿Cuándo se llama Crecer?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dentro de INSERTAR, justo antes de agregar el nodo nuevo

$$\text{si } \frac{H.n+1}{|H.buckets|} \geq 0{,}75, \text{ primero } \textsc{Crecer}(H).$$

Después de CRECER, *H.buckets* mide el doble, y el resto de INSERTAR continúa normalmente (el bucket del nuevo nodo se recalcula con el *H.hash* ya actualizado).

Complejidad amortizada de Crecer

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Igual argumento que `vector`

CRECER cuesta $\Theta(m)$ (recorrer y reubicar todos los nodos), pero **duplicando** m cada vez, esto ocurre cada vez con menos frecuencia relativa a n.

Complejidad amortizada de Crecer

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Igual argumento que `vector`

CRECER cuesta $\Theta(m)$ (recorrer y reubicar todos los nodos), pero **duplicando** m cada vez, esto ocurre cada vez con menos frecuencia relativa a n.

El costo total de todos los CRECER a lo largo de n inserciones es $O(n)$ — es decir, $O(1)$ **amortizado** por inserción, exactamente el mismo análisis que ya vimos para el crecimiento de `vector`.

En la práctica: `std::unordered_map`

La versión de la STL

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En la práctica, no implementas la tuya

C++ trae `std::unordered_map<K,V>`: acceder o **insertar** con `H[clave]`, verificar existencia con `H.count(clave)`, **eliminar** con `H.erase(clave)` — todo en $O(1)$ esperado.

La versión de la STL

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En la práctica, no implementas la tuya

C++ trae `std::unordered_map<K,V>`: acceder o **insertar** con `H[clave]`, verificar existencia con `H.count(clave)`, **eliminar** con `H.erase(clave)` — todo en $O(1)$ esperado.

Por dentro, `std::unordered_map` es exactamente esto — una tabla hash con encadenamiento (en la mayoría de implementaciones), con función hash, factor de carga máximo y crecimiento automático ya resueltos por la biblioteca.

¿Cuándo implementar la propia, entonces?



¿Cuándo implementar la propia, entonces?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- En la práctica, **casi nunca**: usa `unordered_map` (C++), `dict` (Python), o el equivalente de tu lenguaje. **En este curso sí será necesario usar tu hash table propia**.

¿Cuándo implementar la propia, entonces?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- En la práctica, **casi nunca**: usa `unordered_map` (C++), `dict` (Python), o el equivalente de tu lenguaje. **En este curso sí será necesario usar tu hash table propia**.
- Entender **cómo** funciona por dentro te ayuda a predecir su complejidad real, y a diagnosticar cuándo su rendimiento se degrada (por ejemplo, con una función hash mala para tus claves, o un adversario que fuerza colisiones a propósito).

Aplicación: conteo de frecuencias

Ejemplo: contar palabras de un texto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 5: CONTAR-PALABRAS(texto)

$H \leftarrow$ tabla hash vacía
para cada *palabra w en texto* **hacer**
    $v \leftarrow$ BUSCAR(H, w)
    si $v =$ NO-ENCONTRADO **entonces**
        INSERTAR$(H, w, 1)$
    en otro caso
        INSERTAR$(H, w, v + 1)$
devolver H `// H mapea cada palabra a su frecuencia`

Ejemplo: contar palabras de un texto

Algoritmo 6: CONTAR-PALABRAS(texto)

```
H ← tabla hash vacía
para cada palabra w en texto hacer
    v ← BUSCAR(H, w)
    si v = NO-ENCONTRADO entonces
        INSERTAR(H, w, 1)
    en otro caso
        INSERTAR(H, w, v + 1)
devolver H                    // H mapea cada palabra a su frecuencia
```

Cada palabra se procesa en $O(1)$ esperado: leer todo el texto de n palabras cuesta $O(n)$ en total — comparado con $O(n \lg n)$ si tuvieras que ordenar las palabras primero para contarlas.

Tabla resumen: complejidad lograda

	Buscar	Insertar	Eliminar

Tabla resumen: complejidad lograda

	Buscar	**Insertar**	**Eliminar**
Arreglo, sin ordenar	$O(n)$	$O(1)$	$O(n)$

Tabla resumen: complejidad lograda

	Buscar	**Insertar**	**Eliminar**
Arreglo, sin ordenar	$O(n)$	$O(1)$	$O(n)$
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$

Tabla resumen: complejidad lograda

	Buscar	**Insertar**	**Eliminar**
Arreglo, sin ordenar	$O(n)$	$O(1)$	$O(n)$
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$
Lista enlazada	$O(n)$	$O(1)$	$O(n)$

Tabla resumen: complejidad lograda

	Buscar	**Insertar**	**Eliminar**
Arreglo, sin ordenar	$O(n)$	$O(1)$	$O(n)$
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$
Lista enlazada	$O(n)$	$O(1)$	$O(n)$
Tabla hash	$O(1)$ esp.	$O(1)$ am.	$O(1)$ esp.

Tabla resumen: complejidad lograda

	Buscar	**Insertar**	**Eliminar**
Arreglo, sin ordenar	$O(n)$	$O(1)$	$O(n)$
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$
Lista enlazada	$O(n)$	$O(1)$	$O(n)$
Tabla hash	$O(1)$ esp.	$O(1)$ am.	$O(1)$ esp.

La tabla hash es la **única** estructura que vimos con las tres operaciones en $O(1)$ (esperado/amortizado) — el costo es que no mantiene ningún **orden**: recorrer las claves ordenadamente sigue necesitando ordenar aparte.

Resumen de la sesión



Resumen de la sesión

- Una tabla hash con encadenamiento es un `vector` de listas enlazadas — `insertar`/`buscar`/ `eliminar` recorren un solo bucket, ubicado con `hash(clave)`.

Resumen de la sesión

- Una tabla hash con encadenamiento es un `vector` de listas enlazadas — `insertar`/`buscar`/ `eliminar` recorren un solo bucket, ubicado con `hash(clave)`.
- CRECER (duplicar m y reubicar todo) mantiene α acotado, dando $O(1)$ **amortizado** por operación — el mismo patrón que `vector`.

Resumen de la sesión

- Una tabla hash con encadenamiento es un `vector` de listas enlazadas — `insertar`/`buscar`/ `eliminar` recorren un solo bucket, ubicado con `hash(clave)`.
- CRECER (duplicar m y reubicar todo) mantiene α acotado, dando $O(1)$ **amortizado** por operación — el mismo patrón que `vector`.
- En la práctica se usa `unordered_map` (o el equivalente del lenguaje), no una implementación propia — pero entender su funcionamiento interno importa para predecir su rendimiento real.

Resumen de la sesión

- Una tabla hash con encadenamiento es un `vector` de listas enlazadas — `insertar`/`buscar`/ `eliminar` recorren un solo bucket, ubicado con `hash(clave)`.
- CRECER (duplicar m y reubicar todo) mantiene α acotado, dando $O(1)$ **amortizado** por operación — el mismo patrón que `vector`.
- En la práctica se usa `unordered_map` (o el equivalente del lenguaje), no una implementación propia — pero entender su funcionamiento interno importa para predecir su rendimiento real.
- Aplicación típica: contar frecuencias, detectar duplicados, o cualquier tarea que necesite "¿ya vi esta clave antes?" en $O(1)$ esperado.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias